.UBAfiuba
FACULTAD DE INGENIERÍA

Teoría de Algoritmos
(75.29) Curso Buchwald - Genender

# Trabajo Práctico 3
# Problemas NP-Completos

24 de noviembre de 2023

Geronimo Palavecino
109681

Macarena Vita Sanchez
109377

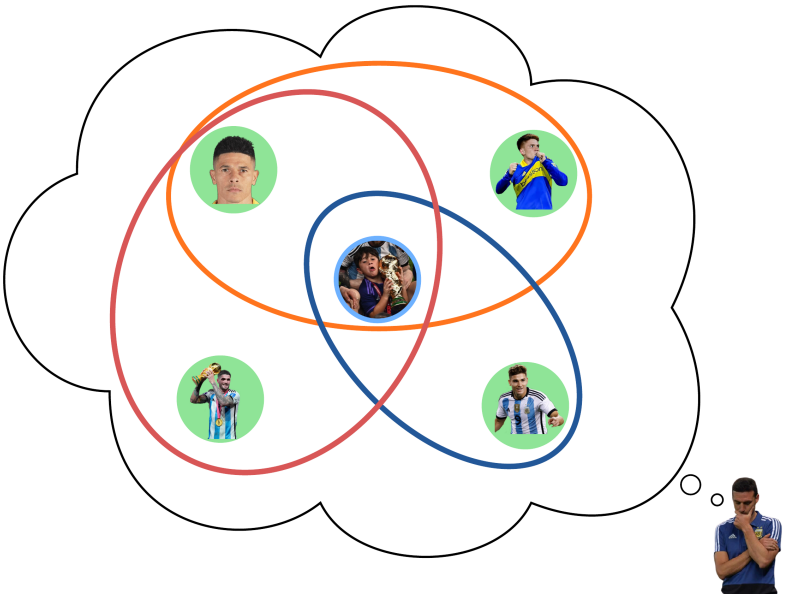
Mariana Galdo Martinez
105658

# 1. Introducción

En este informe, nos sumergimos en el desafío estratégico que enfrenta el director técnico de la selección nacional, Lionel Scaloni. Ante la presión mediática y la necesidad de formar un equipo que contente a diversos sectores, Scaloni se encuentra ante la tarea de seleccionar el conjunto de jugadores para un próximo partido amistoso contra Burkina Faso. A medida que la prensa expresó sus preferencias por distintos jugadores, Scaloni busca encontrar una solución eficiente para conformar un equipo que no solo satisfaga a los diversos medios, sino que también le permita probar tácticas y estrategias específicas.

En esta situación, Scaloni recurre a la asesoría de Carlos Bilardo, un experto en fútbol argentino, quien identifica el problema como un caso particular del conocido "Hitting-Set Problem". Este problema plantea la búsqueda de un subconjunto mínimo de elementos que intersecte con cada uno de varios subconjuntos dados. En el contexto de la selección argentina, los jugadores convocados representan el conjunto universal, los deseos de la prensa se convierten en los subconjuntos, y el conjunto óptimo a seleccionar para el partido amistoso se traduce en el "Hitting Set".

Con la claridad proporcionada por Bilardo, Scaloni busca nuestra asistencia para determinar la eficiencia, en términos polinomiales, de obtener este conjunto deseado. Exploraremos posibles enfoques y soluciones que le permitan a Scaloni tomar decisiones informadas y estratégicas en la formación del equipo para el próximo encuentro contra Burkina Faso.



# Resolución

## 2. Análisis del Problema

### Objetivo Principal

El objetivo principal en este problema es seleccionar el conjunto mínimo de jugadores, $C$, para conformar el equipo que jugará contra Burkina Faso en el próximo partido amistoso. Este conjunto debe cumplir con dos condiciones clave: satisfacer las preferencias de la prensa, representadas por subconjuntos específicos $B_i$, y permitir a Scaloni realizar pruebas y ajustes tácticos durante el encuentro.

### Variables en Juego

1. **Jugadores Convocados ($A$):** Este conjunto representa todos los jugadores disponibles para la convocatoria al equipo nacional. Cada jugador es una variable potencial en la formación del equipo.
2. **Preferencias de la Prensa ($B_i$):** Cada medio o periodista tiene su conjunto de jugadores preferidos para integrar el equipo titular. Estos conjuntos representan las preferencias individuales de la prensa y constituyen los subconjuntos del "Hitting-Set Problem".
3. **Conjunto Óptimo de Jugadores ($C$):** Esta es la variable que buscamos determinar. Representa el subconjunto mínimo de jugadores que intersecta con cada preferencia de la prensa ($B_i$). En términos del "Hitting-Set Problem", $C$ es el hitting set que satisface todas las condiciones impuestas por la prensa.

### Restricciones

Cada preferencia de la prensa ($B_i$) debe ser tocada por al menos un jugador en el conjunto $C$.

$$C \cap B_i \neq \emptyset \quad \forall i$$

# 3. Hitting Set se encuentra en NP

Para verificar que el Hitting-Set Problem está en NP, podemos dividir su verificación en dos partes:

- $|C| \leq K$
- $C$ tenga al menos un elemento de cada $B_i$ $(C \cap Bi \neq \emptyset \quad \forall i)$

La primera verificación se puede hacer en tiempo constante. Para la segunda verificación, tenemos que iterar por todos los subconjuntos $B_i$. Por cada subconjunto, iteramos por todos sus elementos para comprobar si alguno de sus elementos se encuentra en C. En el peor de los casos, si n es la cantidad total de elementos en el conjunto A, $B_i$ podría tener n-1 elementos, y es posible que no encuentre un elemento en común hasta que se haya llegado hasta el último elemento. Por lo tanto, la complejidad temporal para encontrar un valor en común entre un $B_i$ y $C$ es $O(n-1)$. Entonces, para una $m$ cantidad de subconjuntos $B_i$, obtenemos la siguiente complejidad temporal:

$$T(n) = O(m \cdot (n-1)) = O(m \cdot n)$$

$\therefore$ Se puede realizar la verificación del Hitting-Set Problem en tiempo polinomial.

De esta manera, queda demostrado que el Hitting-Set Problem se encuentra en NP.

# 4. Hitting Set es NP Completo

En esta sección, abordaremos la demostración de que el Problema de Hitting Set pertenece a la clase de problemas NP-completos. Para lograr esto, reduciremos un problema NP-completo conocido: **Vertex Cover** al Problema de Hitting Set.

## Problema de Decisión de Vertex Cover

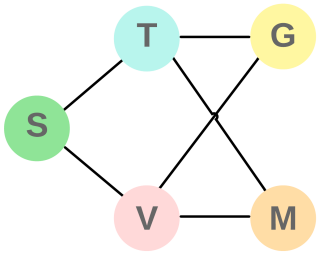
El Problema de Decisión de Vertex Cover se define de la siguiente manera: dado un grafo no dirigido $G = (V, E)$ y un número entero $k$, se busca determinar si existe un conjunto de vértices $C \subseteq V$ tal que $|C| \leq k$ y cada arista en el grafo tiene al menos un extremo en $C$. Formalmente, buscamos verificar la existencia de $C$ cumpliendo:

$$\forall (u, v) \in E, \quad u \in C \quad \vee \quad v \in C$$

## Reducción de Vertex Cover a Hitting Set

Consideremos una instancia $G = (V, E)$ del Problema de Decisión de Vertex Cover con un parámetro $k$. Queremos demostrar que esta instancia se puede reducir de manera polinomial al Problema de Hitting Set.

Definimos $A = V$ como el conjunto universal, y para cada arista $(u, v) \in E$, creamos un subconjunto $B_{(u,v)} = \{u, v\}$ de $A$. El conjunto $A$ y los subconjuntos $B_{(u,v)}$ constituyen la instancia de Hitting Set. Podemos observar este paso en la siguiente figura:




```
A = { T, G, M, V ,S}
B = {{S,T}, {S,V}, {T, G}, {T, M}, {V, G}, {V, M}}
```

Figura 1: Reducción de Vertex Cover a Hitting Set

La pregunta ahora es: ¿existe un subconjunto $C \subseteq A$ con $|C| \leq k$ tal que $C \cap B_{(u,v)} \neq \emptyset$ para todo $(u, v) \in E$?

Para el ejemplo dado, si nosotros solicitamos un Hitting Set de $|C| \leq 3$, nos devolverá que es posible, en cambio, si le pedimos un $k$ menor nos devolverá que no. Esto es aplicable a Vertex Cover dado que no es posible encontrar una cantidad de vértices menor a 3 que cubra todas las aristas.

Esta reducción es polinómica, ya que construir la instancia de Hitting Set puede realizarse en tiempo polinómico con respecto al tamaño de la instancia original de Vertex Cover (Construir $A$ es $O(V)$ y construir los $B_i$ es $O(E)$, resultando en una complejidad temporal de $O(V + E)$)

La existencia de un conjunto de vértices que cubre todas las aristas en $G$ es equivalente a la existencia de un hitting set $C$ en la instancia de Hitting Set de tamaño $k$. Por lo tanto, hemos demostrado que el Problema de Hitting Set es NP-completo.

# 5. Resolución por Backtracking

## Idea General

Para solucionar el problema utilizando backtracking, se plantea explorar recursivamente las posibles combinaciones de elementos que formen un hitting set, utilizando la estrategia de agregar y quitar elementos, tomando decisiones informadas para alcanzar las solución óptima.

## Resolución Algorítmica

La función `backtrackHittingSet` se puede dividir en 3 partes: La parte de verificación, la parte de poda, y la parte de "fuerza bruta".

En la primera parte, se verifica si la solución potencial actual es una solución valida, verificando si faltan subconjuntos por cubrir. Luego, verifica si todavía no se encontró una solución al problema, o si la nueva solución encontrada es mejor que la mejor solución anterior, devolviendo la nueva solución en caso afirmativo; y devolviendo la mejor solución anterior, en caso negativo. Asimismo, si sigue habiendo subconjuntos para cubrir, se verifica si la solución potencial tiene la misma o mayor cantidad de elementos que la mejor solución actual, podando el análisis, y se devuelve la mejor solución hasta ahora. Todas estas verificaciones se realizan en tiempo constante.

En la segunda parte, se considera, si el subconjunto a analizar ya está cubierto por algún elemento de la solución potencial actual, en cuyo caso, realiza un llamado recursivo a la función, pasando al siguiente subconjunto. Para verificar esto, se deben recorrer todos los elementos existentes en el subconjunto, siendo estos en el peor de los casos n-1 elementos, provocando que la verificación sea O(n)

Luego, en la tercera parte, en caso de que el subconjunto a analizar todavía no estuviese cubierto, se consideran de a uno los elementos contenidos en el mismo, agregándolos a la solución potencial y realizando un llamado recursivo para el siguiente subconjunto, siempre guardando el valor de la mejor solución obtenida hasta ahora, antes de quitar el elemento de la solución potencial y pasar al siguiente elemento.

Finalmente, luego de realizar todo esto, se debe haber encontrado la mejor solución posible para este punto con la combinación de elementos dada, por lo que se devuelve dicha mejor solución, y se regresa en la recursión.

Debido a esto, se puede observar que este algoritmo, en el peor de los casos, para cada subconjunto m, realiza n operaciones, provocando que la complejidad resultante sea $O(n^m)$, donde n es la cantidad de elementos en el conjunto A, y m es la cantidad de subconjuntos B.

```python
def backtrackHittingSet(subconjuntos, posibleC, mejorSolucion):
    if not subconjuntos:
        if not mejorSolucion or len(posibleC) < len(mejorSolucion):
          return posibleC.copy()
        return mejorSolucion
    if mejorSolucion and len(posibleC) >= len(mejorSolucion):
        return mejorSolucion

    if any(elemento in posibleC for elemento in subconjuntos[0]):
        return backtrackHittingSet(subconjuntos[1:], posibleC, mejorSolucion)

    for elemento in subconjuntos[0]:
        posibleC.add(elemento)
        mejorSolucion = backtrackHittingSet(subconjuntos[1:], posibleC,
    mejorSolucion)
        posibleC.remove(elemento)

    return mejorSolucion
```

No obstante, se posee una segunda función `solucionBacktracking`, la cual funciona como máscara, para inicializar de forma constante, el set de solución posible, y realizar el primer llamado a la función principal.

```python
def solucionBacktracking(subconjuntos):
    mejorSolucion = backtrackHittingSet(subconjuntos, set(), [])
    return mejorSolucion
```

De esta forma la complejidad total de nuestro algorítmo es $O(n^m)$.

## Mediciones temporales

A continuación se presentan mediciones de los tiempos en los que tarda el algoritmo en llegar a la solución óptima, obtenidas a partir de la generación de elementos pseudoaleatorios. Se puede observar como el algoritmo posee una tendencia exponencial que coincide con la complejidad calculada anteriormente.




Figura 2: Backtracking con 25 iteraciones

# 6. Resolución por Programación Lineal Entera

## Idea general

Para solucionar el problema del Hitting Set utilizando programación lineal, se puede plantear la creación de una variable por cada uno de los elementos del conjunto A, y representar a cada uno de los subconjuntos a cubrir con una inecuación que sume todos sus elementos contenidos, y que esta sumatoria deba ser mayor o igual a uno. Finalmente, para la función objetivo, se puede plantear que la sumatoria de todas las variables sea mínima.

### Variables

La variable $C_i \, , \forall i < n$ se define como:

$$C_i = \begin{cases} 1 & \text{si está en el subconjunto C,} \\ 0 & \text{si no está en el subconjunto C.} \end{cases} \tag{1}$$

### Función Objetivo

$$Z(MIN) = \sum_{i=0}^{n} C_i \tag{2}$$

### Restricciones

En general, las restricciones serían:

$$1 \leq \sum_{C_i \in B} C_i, \quad \forall B \in A \tag{3}$$

Por o que si un elemento está cubierto (por ejemplo para el elemento 2) la ecuación quedaría:

$$1 \leq C_1 + C_2 + C_3 \tag{4}$$

## Resolución algorítmica

La función `crearVariables` itera sobre los elementos de un subconjunto y crea variables de programación lineal para cada elemento, en tiempo contante. En el peor de los casos, para un subconjunto b, hay n-1 elementos, provocando que la complejidad temporal de la función sea O(n)

```
1 def crearVariables(subset, elementosContemplados, relajado):
2     for elemento in subset:
3         if elemento not in elementosContemplados:
4             if relajado:
5                 elementosContemplados[elemento] = pulp.LpVariable(name = str(
    elemento), lowBound = 0, upBound = 1, cat='Continuous')
6             else:
7                 elementosContemplados[elemento] = pulp.LpVariable(name = str(
    elemento), cat='Binary')
```

La función `solucionProgramacionLineal` define el problema de programación lineal a resolver, y utiliza la función `crearVariables` m veces, para crear las variables de los m subconjuntos y su respectiva inecuación, realizándolo en $O(m \cdot n)$.

A continuación, se define en tiempo constante la función de optimización, y se resuelve el problema de programación lineal. Para la librería utilizada en el trabajo, se investigó que el método de resolución utilizado es COIN-OR Branch and Cut Solver, el cual es un método de ramificación

y poda con complejidad temporal $O(b^d)$, donde b es el factor de ramificación y d es la profundidad máxima de los arboles de búsqueda. No obstante, no se logró encontrar los valores default utilizados por la librería para estas variables.

Sin embargo, debido a la naturaleza de la programación lineal respecto de sus variables, sabemos que al trabajar con variables continuas, el método de resolución se comportará de forma polinómica, mientras que al utilizar variables enteras, se comportará de forma exponencial.

Finalmente, el algoritmo incluye en la solución todas las variables que sean mayores a un umbral determinado (En el caso de programación lineal entera, deben ser mayores a $\frac{1}{2}$, y en el caso de programación lineal continua, deben ser mayores a $\frac{1}{b^*}$; donde $b^*$ es la cantidad de elementos del mayor subconjunto)

```
def solucionProgramacionLineal(B, relajado = False):
    problema = pulp.LpProblem("sum", pulp.LpMinimize)
    elementosContemplados = {}
    for subset in B:
      crearVariables(subset, elementosContemplados, relajado)
      expresion = pulp.lpSum(elementosContemplados[elemento] for elemento in subset
    )
      problema += expresion >= 1
    objectivo = pulp.lpSum(elementosContemplados.values())
    problema += objectivo
    problema.solve(pulp.apis.PULP_CBC_CMD(msg=False))
    solucion = set()
    treshold = 1/(len(max(B, key=len))) if relajado else 1/2
    for elemento, variable in elementosContemplados.items():
      if variable.value() >= treshold:
        solucion.add(elemento)
    return solucion
```

De esta forma, se puede concluir que el algoritmo de programación lineal, al utilizar variables enteras, tendrá una tendencia mayor a comportarse de forma exponencial. En cambio, cuando se utilizan variables continuas, tendrá un comportamiento mayormente polinómico.

## Mediciones temporales

A continuación graficamos el comportamiento temporal respecto a la cantidad de elementos del algoritmo de programación lineal




Figura 3: PL con 150 iteraciones

Como se puede apreciar en el gráfico, el algoritmo por Programación Lineal posee una tendencia exponencial. Asimismo, en el gráfico se puede observar la existencia de grandes picos en el tiempo de ejecución. Esto puede ser debido a que, al generar problemas de forma aleatoria, se produzcan problemas peor acondicionados para el algoritmo de resolución de problemas lineales.

# 7. Aproximaciones

## 7.1. Aproximación con Programación Lineal Continua

Como fue mencionado anteriormente, una forma de obtener una solución aproximada al problema, es utilizando el algoritmo de programación lineal. Al mismo se le realiza la modificación de que las variables creadas deben ser continuas dentro del rango [0; 1], y para construir el conjunto solución se toman los elementos que posean un valor mayor a $\frac{1}{b^*}$; siendo $b^*$ la cantidad del subconjunto a cubrir más grande.

### Mediciones temporales

Para poder comprobar de forma empírica la complejidad del algoritmo desarrollado, realizamos mediciones del tiempo que tardaba el algoritmo en resolver problemas con diferentes cantidades de subconjuntos, generados de forma pseudoaleatoria, dando como resultado el siguiente gráfico:




Figura 4: PL Continua con 150 iteraciones




Figura 5: PL Continua con 1000 iteraciones, en pasos de a 100

Como se puede observar, el algoritmo empíricamente posee una tendencia polinómica, en vez de la complejidad exponencial calculada anteriormente para el algoritmo desarrollado con Programación Lineal Entera.

### Análisis de las aproximaciones resultantes

Para analizar qué tan distante puede ser la aproximación encontrada respecto del valor real de la solución óptima, se puede repensar el problema generalizándolo de la siguiente forma:

> Dado un conjunto de elementos $A$ de $n$ elementos, con costos $w_1$, $w_2$, ..., $w_n$ respectivamente, y $m$ subconjuntos $B = b_1, b_2, ..., b_m$ de $A$ ($b_i \subseteq A \forall i$), hallar un subconjunto $C$ que tenga al menos un elemento de cada $b_i$ y que la suma de los costos de sus elementos sea mínima.

Con esta generalización, se debe replantear que la función objetivo de nuestro algoritmo de programación lineal sea $\sum_{a_i \in A} w_i \cdot a_i$. Para esta nueva función objetivo se puede observar que, para Programación Lineal Entera, para cada elemento tomado para la solución, se paga el costo completo del elemento, mientras que para Programación Lineal Continua, se paga entre $\frac{1}{b^*}$ y 1 veces el costo $w_i$ del elemento. Debido a esto, denominando $S^*$ a la solución óptima, y S a la solución aproximada, se pueden plantear la siguiente secuencia de cotas:

$$W(S^*) = \sum_{a_i \in S^*} w_i \cdot a_i \geq \sum_{a_i \in S} w_i \cdot a_i \geq \sum_{a_i \in S} w_i \cdot \frac{1}{b^*} = \frac{1}{b^*} \cdot \sum_{a_i \in S} w_i = \frac{1}{b^*} \cdot W(S) \tag{5}$$

$$W(S^*) \geq \frac{1}{b^*} \cdot W(S) \tag{6}$$

$$b^* \cdot W(S^*) \geq W(S) \tag{7}$$

Asimismo, si se define que todos los elementos del conjunto A poseen un peso igual a 1, el peso de cada una de las soluciones será igual a la cantidad de elementos que poseen, regresando así al problema original. De esta forma, para una instancia I cualquiera del problema original, se pueden definir una solución óptima $z(I) = W(S^*) = |S^*|$, una solución aproximada $A(I) = W(S) = |S|$ y una cota $r(I) = b^*$ tal que:

$$\frac{A(I)}{z(I)} \leq r(I) = b^* \tag{8}$$

### Mediciones de aproximaciones

Para comprobar la cota calculada anteriormente, se generaron instancias del problema con diferentes cantidades de elementos, de forma pseudoaleatoria, y se tomaron mediciones del tamaño de las soluciones óptimas contra el tamaño de las soluciones aproximadas, expuestas en las siguientes imágenes:

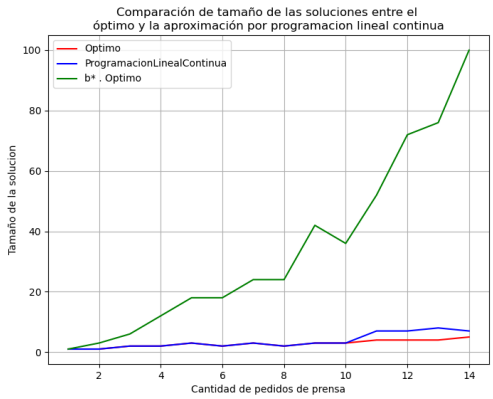



Figura 6: Tamaño de solución óptima vs tamaño de solución aproximada (PL Continua) 15 elementos

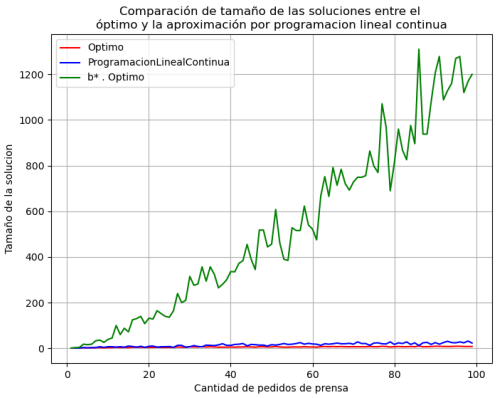



Figura 7: Tamaño de solución óptima vs tamaño de solución aproximada (PL Continua) 100 elementos

Como se puede observar, el tamaño de las aproximaciones obtenidas siempre se encuentra contenida por la cota calculada.

## 7.2. Aproximación Greedy para Hitting Set

La aproximación greedy es una estrategia heurística que toma decisiones locales en cada paso con la esperanza de llegar a una solución global óptima. Aplicada al Problema de Hitting Set, el algoritmo greedy busca seleccionar elementos de manera iterativa para cubrir la mayor cantidad de conjuntos no abarcados.

### Estrategia Greedy

A continuación detallamos la estrategia utilizada:

1. Inicializar $C$ como un conjunto vacío.
2. Mientras haya conjuntos no cubiertos:
   *a*) Seleccionar el elemento que cubre la mayor cantidad de conjuntos no abarcados hasta el momento.

*b*) Agregar ese elemento a $C$.

3. Devolver el conjunto $C$

### Contraejemplo Greedy No Óptimo

Considerar el siguiente conjunto de elementos y subconjuntos:

$$A = \{1, 2, 3, 4, 5\}$$

$$B = \{\{1, 2\}, \{2, 3\}, \{3, 4\}, \{4, 5\}\}$$

Para esta situación, nuestra estrategia greedy puede considerar los elementos **2**, **3** o **4**, debido a que los tres cubren dos subconjuntos. Siendo el peor de los casos aquel en el que selecciona primero al elemento **3**, luego está obligado a seleccionar los elementos **1** o **5**, y **2** o **4**, para poder cubrir el resto de subconjuntos, terminando con una totalidad de tres elementos. No obstante, una solución óptima para esta situación sería seleccionar los elementos **2** y **4**, cubriendo todos los subconjuntos con una totalidad de dos elementos.

### Algorítmo Greedy

A continuación presentamos una implementación de nuestra estrategia greedy en forma de algoritmo:

```
def aproximacionGreedy(subsets):
    elementos = {}
    solucion = set()
    for subset in subsets:
        for element in subset:
            elementos[element] = elementos.get(element, 0) + 1
    setsNoConsiderados = set(map(frozenset, subsets))
    while elementos and setsNoConsiderados:
        element, count = max(elementos.items(), key=lambda x: x[1])
        del elementos[element]
        setsConsiderados = set()
        for subset in setsNoConsiderados:
            if element in subset:
                for e in subset:
                    if e in elementos:
                        elementos[e] -= 1
                setsConsiderados.add(subset)
                solucion.add(element)
        setsNoConsiderados.difference_update(setsConsiderados)
    return solucion
```

La complejidad de este algoritmo depende principalmente del número de elementos y subconjuntos. Supongamos que hay $m$ subconjuntos y en el peor de los casos, cada uno tiene $n - 1$ elementos.

- Recuento de la frecuencia de los `elementos`: $O(m \cdot n)$
- Creación de `setsNoConsiderados`: $O(m)$
- Bucle `while`: $O(m \cdot n)$
- Buscar el máximo: $O(n)$

De esta forma la complejidad total es $O(m \cdot n)$.

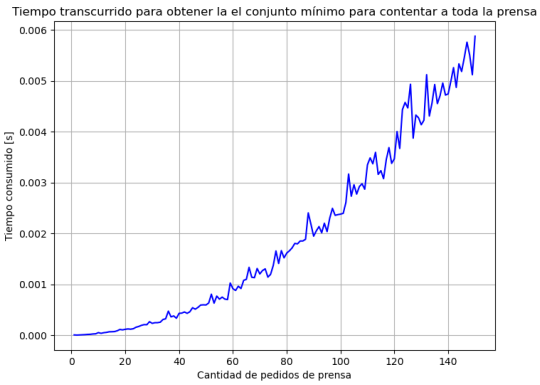



Figura 8: Greedy con 150 iteraciones

### Mediciones temporales

Para corroborar la complejidad previamente calculada, se decidió tomar mediciones del tiempo que tarda el algoritmo en resolver diferentes problemas con elementos generados de forma pesudoaleatoria. Los resultados se pueden observar en la siguiente imagen:

Como se puede apreciar en el gráfico, el algoritmo empíricamente posee una tendencia polinómica.

### Análisis de las aproximaciones resultantes

Para analizar qué tan buena es la aproximación lograda por este algoritmo respecto de la solución óptima, podemos partir de la generalización del problema, anteriormente descripta para Programación Lineal Continua:

> Dado un conjunto de elementos $A$ de $n$ elementos, con costos $w_1$, $w_2$, ..., $w_n$ respectivamente, y $m$ subconjuntos $B = b_1$, $b_2$, ..., $b_m$ de $A$ ($b_i \subseteq A \forall i$), hallar un subconjunto $C$ que tenga al menos un elemento de cada $b_i$ y que la suma de los costos de sus elementos sea mínima.

Considerando esta nueva generalización, se puede observar que una forma de resolver el problema de forma aproximada es:

1. Calcular el costo de cada elemento distribuido entre todos los subconjuntos no considerados hasta ahora, en los que aparece dicho elemento
2. Tomar el elemento con menor costo distribuido
3. Considerar los subconjuntos que abarca
4. Verificar si se consideran todos los subconjuntos $B_i$. En caso negativo, regresar al primer paso; en caso positivo, tomar como solución todos los elementos tomados hasta ahora

Analizando el algoritmo anterior, se puede notar que este es el procedimiento seguido, de forma indirecta, por el algoritmo greedy previamente planteado. De esta forma, se puede definir para el primer paso del algoritmo, nuevas variables que representen el costo de un elemento distribuido entre los subconjuntos cubiertos por dicho elemento, en la iteración en la que se lo incluyó:

$$c_{b_i} = \frac{w_a}{\sum_{\forall b_i \in (B \cap R) | a_i \in b_i} 1}$$; R representa el conjunto de subconjuntos no considerados hasta el momento (9)

Como para cada uno de los elementos seleccionados, su costo es distribuido entre los nuevos subconjuntos cubiertos, se puede definir que la sumatoria de los costos distribuidos suman el costo total de formar el set solución. Llamando S a la solución encontrada por el algoritmo greedy:

$$\sum_{a_i \in S} w_i = \sum_{b_i \in B} c_{b_i} \tag{10}$$

Ahora, analizando para un elemento $a_i$ genérico, el cual cubre los primeros $d$ elementos de $B$, y asumiendo que estos se encuentran en el orden en el que se les asignan los costos $c_{b_j}$. Considerando un caso en el cual se está en la iteración k ($k \leq d$) para la cual es considerado el subconjunto $b_k$. Para esta iteración quedan como máximo $d - k + 1$ subconjuntos nuevos por cubrir, pudiendo así acotar el costo distribuido de los subconjuntos de la siguiente forma:

$$\frac{w_{a_i}}{\sum_{\forall b_i \in (B \cap R) | a_i \in b_i} 1} \leq \frac{w_{a_i}}{d - k + 1} \tag{11}$$

Asimismo, sabemos que para esta iteración, por la regla utilizada por el algoritmo greedy, se toma el elemento $a_h$ que genere el menor costo distribuido, el cual debe ser menor o igual al costo distribuido de $a_i$, resultando de esta forma:

$$c_{b_j} = \frac{w_{a_h}}{\sum_{\forall b_i \in (B \cap R) | a_i \in b_i} 1} \leq \frac{w_{a_i}}{\sum_{\forall b_i \in (B \cap R) | a_i \in b_i} 1} \leq \frac{w_{a_i}}{d - k + 1} \tag{12}$$

Con esto, si se aplica la sumatoria a todos los subsets abarcados por el elemento $a_i$:

$$\sum_{\{\forall b | a_i \in b\}} c_b = \sum_{k=1}^{d} c_{b_j} \leq \sum_{k=1}^{d} \frac{w_{a_i}}{d - k + 1} = \frac{w_{a_i}}{d} + \frac{w_{a_i}}{d-1} + ... + \frac{w_{a_i}}{1} = w_{a_i} \cdot (\frac{1}{d} + \frac{1}{d-1} + ... + \frac{1}{1}) = w_{a_i} \cdot H(d) \tag{13}$$

De esta forma se puede acotar que para cada elemento $a_i$, la suma de los pesos distribuidos de los subconjuntos que cubre es como máximo $H(d)$ veces $w_{a_i}$ (Siendo $d$ la cantidad de subconjuntos que cubre el elemento). Esto es lo mismo que decir que $w_{a_i}$ es mayor o igual a la suma de los pesos distribuidos de los subconjuntos, reducida $H(d)$ veces:

$$w_{a_i} \geq \frac{1}{H(d)} \cdot \sum_{\{\forall b | a_i \in b\}} c_b \tag{14}$$

Finalmente, sea $S^*$ una solución óptima para el problema , de forma tal que $w^* = \sum_{a_i \in S^*} w_{a_i}$. Si tomamos una $d^*$ tal que sea igual a la longitud del subset que contenga mayor cantidad de elementos, se puede definir:

$$w^* = \sum_{a_i \in S^*} w_{a_i} \geq \sum_{a_i \in S^*} \frac{1}{H(d^*)} \cdot \sum_{\{\forall b | a_i \in b\}} c_b \geq \frac{1}{H(d^*)} \cdot \sum_{b \in B} c_b = \frac{1}{H(d^*)} \cdot \sum_{a_i \in S} w_{a_i} \tag{15}$$

Por último, se puede acotar la función armónica por $ln(d^*)$. De esta forma, si se toman los costos para todos los elementos iguales a 1 (Regresando al problema original) queda en evidencia, que la aproximación lograda por el algoritmo greedy es como máximo $log(d^*)$ veces mayor que la solución óptima, para una instancia cualquiera I del problema, queda:

$$\frac{A(I)}{z(I)} \leq r(I) = log(d^*) \tag{16}$$

### Mediciones de aproximaciones

Para verificar que la cota calculada anteriormente sea correcta, se generaron problemas con diferentes cantidades de elementos, de forma pseudoaleatoria, y se tomaron mediciones del tamaño de las soluciones óptimas y de las soluciones aproximadas. Los resultados se exponen en los siguientes gráficos:

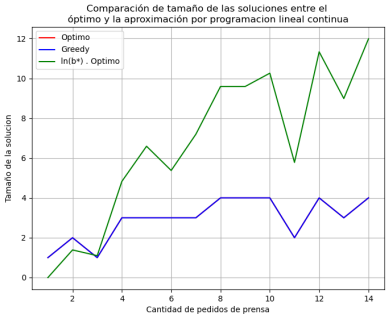



Figura 9: Tamaño de solución óptima vs tamaño de solución aproximada (Greedy) 15 elementos

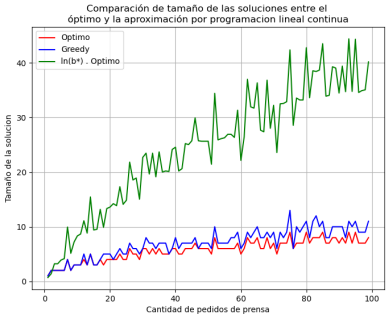



Figura 10: Tamaño de solución óptima vs tamaño de solución aproximada (Greedy) 100 elementos

# 8. Comparación de algoritmos

En esta sección, decidimos realizar comparaciones entre los algoritmos desarrollados, para entender cuales de ellos son los mejores al momento de resolver el problema de Hitting Set

## 8.1. Programación Lineal vs Backtracking

Debido a que tanto el algoritmo de programación lineal como el de Backtracking poseen la misma complejidad temporal, decidimos realizar una comparación de tiempos de ejecución para la resolución de problemas generados con datos aleatorios, para poder decidir empíricamente cual es mas veloz al momento de resolver este tipo de problema Los datos resultantes de la comparación anteriormente descripta se pueden ver reflejados a continuación:

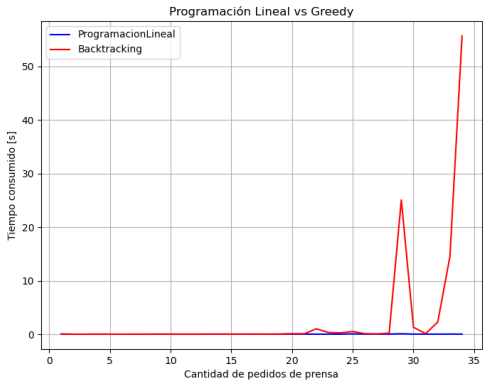



Figura 11: Programación Lineal vs Backtracking

Como se puede observar en el gráfico, el algoritmo de programación lineal posee menor tiempo de ejecución para resolver los problemas de Hitting Set.

## 8.2. Programación Lineal Continua vs Greedy

Para el caso de estos dos algoritmos, no solo se quiere poder decidir cual de los dos funciona más rápidamente al momento de obtener una solución a un problema de Hitting Set dado, sino que también se quiere saber cuál de los dos algoritmos posee soluciones aproximadas de menor tamaño. Para decidir estas cuestiones de forma empírica, se realizaron gráficos, uno considerando las velocidades en las que los algoritmos resolvían diferentes problemas, y otro en el que se consideraban los tamaños de las soluciones aproximadas provistas por los algoritmos. Para ambos casos se crearon problemas de diferentes tamaños, generando elementos de forma pseudoaleatoria:

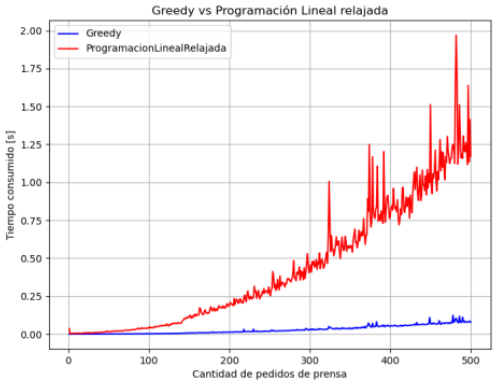



Figura 12: Greedy vs Programación Lineal Continua (Tiempos)

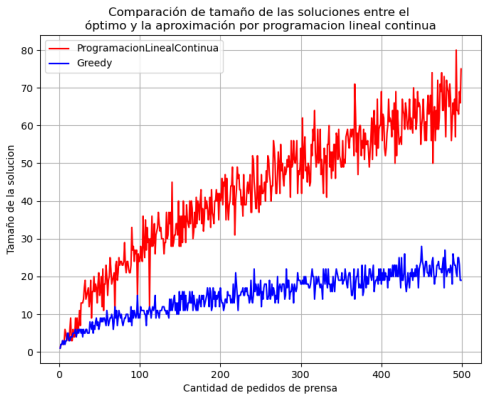



Figura 13: Greedy vs Programación Lineal Continua (Tamaños)

Como se puede observar en los gráficos, el algoritmo greedy posee mejor rendimiento tanto en términos temporales como de aproximación. Debido a esto, si se desea obtener una aproximación para el problema de hitting set problem, es conveniente utilizar el algoritmo greedy por sobre el de programación lineal continua, debido a que este empíricamente , devuelve soluciones más cercanas a las soluciones óptimas, de forma más veloz.

## 8.3. Programación Lineal vs Greedy

En esta situación se quiere analizar que tanta diferencia puede llegar a haber al calcular una solución aproximada o una óptima. Para esto, al igual que para el caso anterior, se tomaron mediciones, para problemas con elementos generados de forma pseudoaleaotira, de las velocidades de resolución, como de los tamaños de las soluciones, resultando en los siguientes gráficos, tomando el mejor algoritmo óptimo y el mejor aproximado:

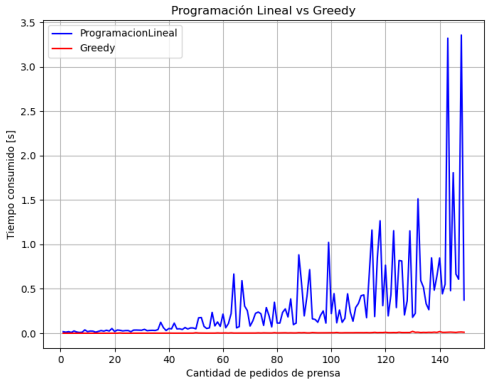



Figura 14: Greedy vs Programación Lineal (Tiempos)

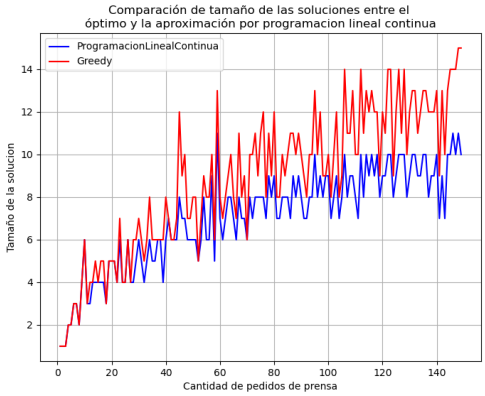



Figura 15: Greedy vs Programación Lineal (Tamaños)

Haciendo una comparativa entre las dos técnicas de diseño se trata de un trade-off entre rapidez y precisión. Aunque el algoritmo greedy ofrece soluciones más rápidas, estas pueden desviarse significativamente de la óptima. Sin embargo, demuestra ser una aproximación eficaz en muchos casos, proporcionando soluciones de calidad aceptable en un tiempo razonable.

# 9. Conclusiones

Inicialmente, se destacó la capacidad de estrategias como Backtracking y Programación Lineal Entera para alcanzar soluciones óptimas, a pesar de sus tiempos de ejecución empíricos notables. Asimismo, se evidenció cómo la aproximación de programación lineal entera presenta la ventaja de mostrar comportamientos polinomiales en determinados escenarios, produciendo así una ventaja al momento de la resolución de problemas, por sobre Backtracking.

Por otro lado, encontramos aproximaciones a la soluciones, realizadas con las estrategias Greedy y Programación Lineal Continua dentro de las cuales se destacó Greedy por su sencillez, tiempos de ejecución empíricos y proporción del tamaño de sus soluciones, respecto de la óptima.

Finalmente, la estrategia Greedy es valiosa cuando se busca un equilibrio entre eficiencia en tiempos de ejecución, y tamaño de la solución. Mientras que, si buscamos precisión, la mejor estrategia para este tipo de problema podría ser la de Programación Lineal Entera.